

JAVASCRIPT

Table of Contents

Sl. No	Topic	Pages
1	Introduction to JavaScript	65-75
2	Conditional Statements	76-79
3	Loops	80-83
4	String	84-87
5	Arrays	88-92
6	Functions	93-98
7	DOM Manipulation	99-108
8	Output Tracing Practice	109-118
9	Code Writing Practice	119-126
10	Fill in the Blanks Practice	127-128
11	MCQ practice	129-138
12	Lecture Video Link	138

INTRODUCTION TO JAVASCRIPT

Introduction

➤ What is JavaScript?

- ⇒ **JavaScript (JS)** is a high-level, dynamic, interpreted programming language.
- ⇒ Primarily used to make web pages **interactive** and **dynamic**.
- ⇒ Runs in the browser but can also run on servers using **Node.js**.

➤ Features of JavaScript?

- **Interpreted:** No need for compilation, runs directly in the browser.
- **Dynamic Typing:** Data types are determined at runtime.
- **Event-Driven:** Can respond to user actions (clicks, keystrokes, etc.).
- **Prototype-Based:** Uses prototypes for inheritance instead of classical classes.
- **Cross-Platform:** Works on all major browsers and devices.

Difference Between JavaScript, Java, and ECMAScript

Feature	JavaScript	Java	ECMAScript
Type	Scripting language	Object-oriented programming	Standard specification for JS
Execution	Runs in browsers & Node.js	Runs in JVM	Not a language, but a standard
Syntax	Loosely typed	Strongly typed	Defines syntax & rules of JavaScript
Usage	Web interactivity	Enterprise applications	Acts as blueprint for JavaScript

➤ History

- ⇒ Created in **1995** by **Brendan Eich** at Netscape.
- ⇒ Originally called **Mocha**, then **LiveScript**, and finally **JavaScript**.
- ⇒ Standardized as **ECMAScript** in 1997 by **ECMA International**.

➤ Why Learn JavaScript?

- **Essential for Web Development:** Alongside HTML & CSS, it's one of the three core web technologies.
- **Versatile:** Works for frontend, backend, mobile apps, desktop apps, and even IoT.
- **Huge Community & Libraries:** jQuery, React, Vue, Angular, Node.js, Express.js.

Variables

⇒ **Variable** is a name used to **store data** in memory. It acts as a container for values.

➤ Declaring Variables?

Keyword	Scope	Reassignment	Hoisting	Notes
var	Function	Yes	Yes	Old way, avoid in modern JS
let	Block	Yes	No	Preferred for variables that change
const	Block	No	No	Preferred for constants

➤ How to Declare a Variable?

Syntax:	Example Code:
<pre>var x = 10; let y = 20; const z = 30;</pre>	<pre>var x = 10; // function-scoped let y = 20; // block-scoped const z = 30; // cannot be reassigned</pre>
Rules for Naming Variables:	
• Must start with a letter or underscore (_) • Cannot start with a digit • Can contain letters, digits, and underscores • Case-sensitive (age and Age are different)	

Data Types

⇒ JavaScript has **primitive types** and **reference types**.

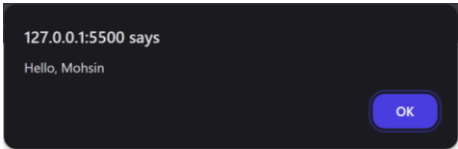
Primitive Types:		
Type	Example	Description
string	"Hello"	Text data
number	42, 3.14	Numbers (integer & float)
boolean	true, false	Logical values
null	null	Intentional empty value
undefined	let x;	Declared but not assigned
symbol	Symbol('id')	Unique identifiers
bigint	123n	Large integers

Example Code:

```
let name = "Alice";    // string
let age = 20;          // number
let isStudent = true;  // boolean
let car = null;        // null
let city;              // undefined
```

Output Methods

Method	Where It Shows	Example
<code>console.log()</code>	Browser console	<code>console.log("Hello")</code>
<code>alert()</code>	Popup box	<code>alert("Warning!")</code>
<code>document.write()</code>	Directly to web page	<code>document.write("Hi")</code>

Example Code: <code>console.log()</code>	Output:
<pre>let name_ = "Mohsin"; console.log("Name:", name_)</pre>	Mohsin
Example Code: <code>alert()</code>	Output:
<pre>let name_ = "Mohsin"; alert("Hello, " + name_);</pre>	
Example Code: <code>document.write()</code>	Output:
<pre>let name_ = "Mohsin"; const country = "Bangladesh"; let age = 21; document.write(`You are from \${country} and \${age} years old.`);</pre>	You are from Bangladesh and 21 years old.

How to Run JavaScript

In HTML:

```
<!-- Inline -->
<script>
  console.log("Hello World");
</script>

<!-- External File -->
<script src="script.js"></script>
```

Tip: Place scripts **before** `</body>` so the page loads faster.

Type Conversion

➔ **Type Conversion** means changing the data type of a value or variable.

Conversion	Example	Output
String → Number	<code>Number("10")</code>	10
Number → String	<code>String(10)</code>	"10"
Boolean → Number	<code>Number(true)</code>	1
Automatic (coercion)	<code>"5" * 2</code>	10

Comments

⇒ **Comments** are notes in the code that are ignored by the browser, used to explain or temporarily disable code.

Type	Syntax	Example
Single-line	<code>//</code>	<code>// This is a comment</code>
Multi-line	<code>/* ... */</code>	<code>/* This is a multi-line comment */</code>

Operators

⇒ Operators are **special symbols** used to perform operations on **values or variables**.

Example Code:	Output:
<pre>let a = 5; let b = 2; let sum = a + b; // + is the operator console.log(sum); // 7</pre>	7

Types of Operators

a) Arithmetic Operators:

⇒ Used for **mathematical calculations**.

Operator	Description	Example	Result
+	Addition	5 + 2	7
-	Subtraction	5 - 2	3
*	Multiplication	5 * 2	10
/	Division	5 / 2	2.5
%	Modulus (remainder)	5 % 2	1
**	Exponentiation	5 ** 2	25
++	Increment (+1)	a++	6
--	Decrement (-1)	b--	1

b) Assignment Operators:

⇒ Used to **assign values** to variables.

Operator	Example	Meaning
=	x = 5	Assign 5 to x
+=	x += 2	x = x + 2
-=	x -= 2	x = x - 2
*=	x *= 2	x = x * 2
/=	x /= 2	x = x / 2
%=	x %= 2	x = x % 2

c) Comparison Operators:

⇒ Used to **compare two values**. Result is **true** or **false**.

Operator	Example	Meaning
<code>==</code>	<code>5 == '5'</code>	Equal (value only)
<code>===</code>	<code>5 === '5'</code>	Equal (value & type)
<code>!=</code>	<code>5 != 2</code>	Not equal (value)
<code>!==</code>	<code>5 !== '5'</code>	Not equal (value & type)
<code>></code>	<code>5 > 2</code>	Greater than
<code><</code>	<code>5 < 2</code>	Less than
<code>>=</code>	<code>5 >= 5</code>	Greater or equal
<code><=</code>	<code>5 <= 2</code>	Less or equal

d) Logical Operators:

⇒ Used for **Boolean logic**.

Operator	Example	Result	Meaning
<code>&&</code>	<code>true && false</code>	<code>false</code>	AND — true only if both are true
<code> </code>	<code>true false</code>	<code>true</code>	OR — true if at least one is true
<code>!</code>	<code>!true</code>	<code>false</code>	NOT — flips the Boolean value

In JavaScript:

- `&&` stops checking if the first value is **false** (short-circuiting).
- `||` stops checking if the first value is **true**.
- `!` works on truth/falsie values, not just `true/false`.

e) String Operators:

⇒ Mainly **concatenation**.

Example Code:	Output:
<pre>let firstName = "Mohsin"; let lastName = "ELI"; let fullName = firstName + " " + lastName; console.log(fullName);</pre>	Mohsin ELI

How to See the Data Type

⇒ `typeof` → returns **data type**.

Example Code:	Output:
<pre>let x = 10; let str = "Hello, World!"; console.log(typeof x); // number console.log(typeof str); // string</pre>	<pre>number string</pre>

Keywords

➔ **Keywords** are reserved words in JavaScript. We cannot use them as variable names or identifiers.

List of Common JavaScript Keywords:

break	case	catch	class	const	continue
debugger	default	delete	do	else	export
extends	finally	for	function	if	import
in	instanceof	new	return	super	switch
this	throw	try	typeof	var	void
while	with	yield	let	enum	await

- ✂ **Problem1:** Write a JavaScript program that assign values to variables num1, num2 and perform addition operations. Also print the results.

Code:	Output:
<pre>let num1 = 10; let num2 = 5; let sum = num1 + num2; console.log("Sum:", sum);</pre>	Sum: 15

- ✂ **Problem2:** Write a JavaScript program that assign values to variables num1, num2 and perform subtraction operations. Also print the results.

Code:	Output:
<pre>let num1 = 10; let num2 = 5; let sub = num1 - num2; console.log("Sub:", sub);</pre>	Sub: 5

- ✂ **Problem3:** Write a JavaScript program that assign values to variables num1, num2 and perform multiplication & division operations. Also print the results.

Code:	Output:
<pre>let num1 = 10; let num2 = 5; let mult = num1 * num2; let div = num1 / num2; console.log("Mult:", mult); console.log("Div:", div);</pre>	Mult: 50 Div: 2.0

Input in JavaScript

➔ The `prompt()` shows a pop-up box that asks the user for input.

Syntax:

```
let variable = prompt("Enter something: ")
```

Example Code:

Example code:	Output:
<pre>let name = prompt("Enter your name:"); console.log("Hello, " + name);</pre>	Enter your name: Mohsin Ibna Hossain Hello, Mohsin Ibna Hossain
Convert input to other types:	Output:
<pre>let input = prompt("Enter a number:"); console.log("Type:", typeof input); let num = Number(input); console.log("Number:", num); console.log("Type:", typeof num);</pre>	Enter a number: 23 Type: string Number: 23 Type: number

➔ **Note:** In JavaScript, when you take user input (usually with `prompt()` in the browser), it comes in as a **string**, so you need to convert it to other types like **number** or **boolean** if needed.

Conversion Type	Method	Example	Result
String → Number (integer)	<code>parseInt()</code>	<code>parseInt("42")</code>	42
String → Number (float)	<code>parseFloat()</code>	<code>parseFloat("3.14")</code>	3.14
String → Number (auto)	<code>Number()</code>	<code>Number("42.5")</code>	42.5
Number → String	<code>.toString()</code> OR <code>String()</code>	<code>(42).toString()</code>	"42"
String → Boolean	<code>Boolean()</code>	<code>Boolean("Hello")</code>	true
Boolean → Number	<code>Number(true)</code>	<code>Number(true)</code>	1

Practice Problem

🔗 **Problem1:** Write a Program to input 2 numbers & print their sum.

Code:	Output:
<pre>let num1 = Number(prompt("Enter number 1:")); let num2 = Number(prompt("Enter number 2:")); let sum = num1 + num2; console.log("Sum:", sum);</pre>	Enter number 1: 10 Enter number 2: 20 Sum: 30

🔗 **Problem2:** Write a JavaScript program to take five float variables as input from the user. Find out the summation and average of five numbers

Code:	Output:
<pre>let num1 = parseFloat(prompt("Enter number 1:")); let num2 = parseFloat(prompt("Enter number 2:")); let num3 = parseFloat(prompt("Enter number 3:")); let num4 = parseFloat(prompt("Enter number 4:")); let num5 = parseFloat(prompt("Enter number 5:")); let summ = num1 + num2 + num3 + num4 + num5; let average = summ / 5; console.log("Summation:", summ); console.log("Average:", average);</pre>	Enter number 1: 10 Enter number 2: 3 Enter number 3: 7 Enter number 4: 6 Enter number 5: 8 Summation: 34 Average: 6.8

CONDITIONAL STATEMENTS

Conditional Statements

➔ **Conditional statements** let you execute different blocks of code based on conditions.

Common String Operations:

if Statement:	
<pre>let age = 20; if (age >= 18) { console.log("You are an adult"); }</pre>	
if...else Statement:	if...else if...else Statement:
<pre>let age = 16; if (age >= 18) { console.log("You are an adult"); } else { console.log("You are a minor"); }</pre>	<pre>let score = 85; if (score >= 90) { console.log("Grade: A"); } else if (score >= 80) { console.log("Grade: B"); } else { console.log("Grade: C or below"); }</pre>
Notes: Conditions use comparison operators (==, !=, >, <, >=, <=)	

Nested Conditional Statements

➔ A **nested conditional** means using one if or else block **inside another**.

It allows for **more complex decision making**.

Nested if:	Output:
<pre>let age = 25; let hasID = true; if (age >= 18) { if (hasID) { console.log("You can enter"); } else { console.log("ID required"); } } else { console.log("You are too young"); }</pre>	You can enter

Practice Problem

✂ **Problem1:** Write a JavaScript program to check whether a number entered by the user is **odd** or **even**, and print the result.

Code:	Output:
<pre>let num = Number(prompt("Enter a number:")); if (num % 2 === 0) { console.log("Even"); } else { console.log("Odd"); }</pre>	Enter a number: 10 Even
	Enter a number: 5 Odd

✂ **Problem2:** Write a JavaScript program to **grade students based on their marks** using the following criteria:

- If marks are **90 or above**, grade = "A"
 - If marks are **80 to 89**, grade = "B"
 - If marks are **70 to 79**, grade = "C"
 - If marks are **below 70**, grade = "D"
- Then print the grade.

Code:	Output:
<pre>let marks = Number(prompt("Enter student's marks:")); let grade; if (marks >= 90) { grade = "A"; } else if (marks >= 80) { grade = "B"; } else if (marks >= 70) { grade = "C"; } else { grade = "D"; } console.log("Grade:", grade);</pre>	Enter student's marks: 73 Grade: C

✂ **Problem3:** Write a JavaScript program to input three numbers from the user and print the **greatest** among them.

Code:	Output:
<pre>let a = Number(prompt("Enter first number:")); let b = Number(prompt("Enter second number:")); let c = Number(prompt("Enter third number:")); let greatest; if (a >= b && a >= c) { greatest = a; } else if (b >= a && b >= c) { greatest = b; } else { greatest = c; } console.log("The greatest number is:", greatest);</pre>	Enter first number: 10 Enter second number: 20 Enter third number: 30 The greatest number is: 30
	Enter first number: 15 Enter second number: 10 Enter third number: 5 The greatest number is: 15
	Enter first number: 7 Enter second number: 21 Enter third number: 14 The greatest number is: 21

✂ **Problem6:** Write a JavaScript program to check whether a number is divisible by 2, 5, and 11. Also, demonstrate the use of both OR and AND operators in the program.

Code:	Output:
<pre>let num = Number(prompt("Enter a number:")); if (num % 2 === 0 && num % 5 === 0 && num % 11 === 0) { console.log("Number is divisible by 2, 5, and 11."); } else { console.log("Number is NOT divisible by 2, 5, and 11."); } if (num % 2 === 0 num % 5 === 0 num % 11 === 0) { console.log("Number is divisible by at least one of 2, 5, or 11."); } else { console.log("Number is not divisible by 2, 5, or 11."); }</pre>	Enter a number: 110 number is divisible by 2, 5, and 11. number is divisible by at least one of 2, 5, or 11.
	Enter a number: 37 number is NOT divisible by 2, 5, and 11. number is not divisible by 2, 5, or 11.

LOOPS

Loops

➔ **Loops** are used to execute a block of code repeatedly.

Types of Loops:

for Loop: iterate over a sequence	while Loop: runs while a condition is <code>True</code>
<pre>for (let i = 1; i <= 5; i++) { console.log(i); }</pre>	<pre>let i = 1; while (i <= 5) { console.log(i); i++; }</pre>
do...while Loop: Runs the code once before checking the condition.	for...of Loop: Loops through iterable objects like arrays or strings.
<pre>let i = 1; do { console.log(i); i++; } while (i <= 5);</pre>	<pre>let colors = ["red", "green", "blue"]; for (let color of colors) { console.log(color); }</pre>
for...in Loop: Loops through object properties .	
<pre>let person = {name: "John", age: 30}; for (let key in person) { console.log(key + ": " + person[key]); }</pre>	
break Statement: Stops the loop immediately.	continue Statement: Skips the current iteration and moves to the next.
<pre>for (let i = 1; i <= 10; i++) { if (i === 5) { break; } console.log(i); }</pre>	<pre>for (let i = 1; i <= 5; i++) { if (i === 3){ continue; } console.log(i); }</pre>

Practice Problem

🔗 **Problem1:** Write a JavaScript program to **print numbers from 1 to 5** using a **while** loop.

Code:	Output:
<pre>let i = 1; while (i <= 5) { console.log(i); i++; }</pre>	1 2 3 4 5

🔗 **Problem2:** Write a JavaScript program to **print numbers from 5 to 1** using a **while** loop.

Code:	Output:
<pre>let i = 5; while (i >= 1) { console.log(i); i--; }</pre>	5 4 3 2 1

🔗 **Problem3:** Write a JavaScript program to input a number **n** and print its **multiplication table** using a **while** loop.

Code:	Output:
<pre>let n = parseInt(prompt("Enter a number:")); let i = 1; while (i <= 10) { console.log(n + " x " + i + " = " + (n * i)); i++; }</pre>	Enter a number: 11 11 x 1 = 11 11 x 2 = 22 11 x 3 = 33 11 x 4 = 44 11 x 5 = 55 11 x 6 = 66 11 x 7 = 77 11 x 8 = 88 11 x 9 = 99 11 x 10 = 110

🔗 **Problem4:** Write a JavaScript program to **input a number n** and print the **sum of all numbers from 1 to n** using a **for** loop.

Code:	Output:
<pre>let n = parseInt(prompt("Enter a number:")); let sum = 0; for (let i = 1; i <= n; i++) { sum += i; } console.log("The sum of 1 to " + n + " is: " + sum);</pre>	Enter a number: 10 The sum of 1 to 10 is: 55

✂ **Problem4:** Write a JavaScript program that takes numbers as input from the user using a `while` loop. The loop should **stop if the user enters -1**. Print the **sum** of all entered numbers (excluding -1).

Code:	Output:
<pre>let total = 0; while (true) { let num = parseInt(prompt("Enter a number (-1 to stop):")); if (num === -1) { break; } total += num; } console.log("Total sum is: " + total);</pre>	Enter a number (-1 to stop): 10 Enter a number (-1 to stop): 5 Enter a number (-1 to stop): 3 Enter a number (-1 to stop): 2 Enter a number (-1 to stop): 1 Enter a number (-1 to stop): -1 Total sum is: 21

✂ **Problem5:** Write a JavaScript program to print all numbers from **1 to 10 in a line**, except for number **5**, using a `for` loop and the `continue` statement.

Code:	Output:
<pre>for (let i = 1; i <= 10; i++) { if (i === 5) { continue; } process.stdout.write(i + " "); }</pre>	1 2 3 4 6 7 8 9 10

STRING

Strings

➔ A **string** is a sequence of characters inside **single quotes** ('), **double quotes** (""), or **backticks** (`).

Example Code:

```
let str1 = "Hello";
let str2 = 'World';
let str3 = `Hello World`; // template literal
```

String Creation:

```
let single = 'Hello';
let double = "Hello";
let backtick = `Hello`; // allows variables & expressions
```

Template Literals: Allow variables and expressions inside strings using `${}`.

```
let name = "John";
let age = 25;
console.log(`My name is ${name} and I am ${age} years old.`);
```

Common String Methods:

Method	Description	Example
<code>toUpperCase()</code>	Converts to uppercase	<code>"hello".toUpperCase()</code> → <code>"HELLO"</code>
<code>toLowerCase()</code>	Converts to lowercase	<code>"HELLO".toLowerCase()</code> → <code>"hello"</code>
<code>charAt(index)</code>	Returns character at index	<code>"Hello".charAt(1)</code> → <code>"e"</code>
<code>indexOf(value)</code>	Finds position of first match	<code>"Hello".indexOf("l")</code> → 2
<code>lastIndexOf(value)</code>	Finds last match	<code>"Hello".lastIndexOf("l")</code> → 3
<code>includes(value)</code>	Checks if contains substring	<code>"Hello".includes("lo")</code> → <code>true</code>
<code>startsWith(value)</code>	Checks if starts with substring	<code>"Hello".startsWith("He")</code> → <code>true</code>
<code>endsWith(value)</code>	Checks if ends with substring	<code>"Hello".endsWith("lo")</code> → <code>true</code>

<code>slice(start, end)</code>	Extracts part of a string	<code>"Hello".slice(1, 4) → "ell"</code>
<code>substring(start, end)</code>	Similar to slice	<code>"Hello".substring(1, 4) → "ell"</code>
<code>substr(start, length)</code>	Extracts part by length	<code>"Hello".substr(1, 3) → "ell"</code>
<code>replace(old, new)</code>	Replaces text	<code>"Hello".replace("He", "Ye") → "Yello"</code>
<code>replaceAll(old, new)</code>	Replaces all matches	<code>"hi hi".replaceAll("hi", "hey") → "hey hey"</code>
<code>trim()</code>	Removes spaces from both ends	<code>" hello ".trim() → "hello"</code>
<code>repeat(n)</code>	Repeats string n times	<code>"Hi".repeat(3) → "HiHiHi"</code>
<code>split(separator)</code>	Converts to array	<code>"a,b,c".split(",") → ["a", "b", "c"]</code>

Common String Properties:

Property	Description	Example
<code>.length</code>	Returns string length	<code>"Hello".length // 5</code>

Escape Characters

Code	Meaning
<code>\'</code>	Single quote
<code>\"</code>	Double quote
<code>\\</code>	Backslash
<code>\n</code>	New line
<code>\t</code>	Tab

Example code:	Output:
<pre>console.log("Hello\nWorld");</pre>	Hello World
Example code:	Output:
<pre>let a = "Hello"; let b = "World"; console.log(a + " " + b); console.log(`\${a} \${b}`);</pre>	Hello World Hello World

Practice Problem

🔗 **Problem1:** Write a JavaScript to input user's first name & print its length.

Code:	Output:
<pre>let first_name = prompt("Enter first name:"); let length = first_name.length; console.log("Length of first name is: " + length);</pre>	Enter first name: Mohsin Length of first name is: 6

🔗 **Problem2:** Write a JavaScript program to initialize a string and find how many times the character '\$' appears in it. text = "He bought a pen for \$5 and a book for \$10, total \$15."

Code:	Output:
<pre>let text = "He bought a pen for \$5 and a book for \$10, total \$15."; let count = 0; for (let i = 0; i < text.length; i++) { if (text[i] === '\$') { count++; } } console.log("The character '\$' appears " + count + " time(s).");</pre>	The character '\$' appears 3 time(s).

🔗 **Problem3:** Prompt the user to enter their full name. Generate a username for them based on the input. Start username with @, followed by their full name and ending with the fullname length. eg: user name = "mohsineli", username should be "@mohsineli9"

Code:	Output:
<pre>const fullName = prompt("full name:"); const username = `@\${fullName}\${fullName.length}`; console.log(`Your username: \${username}`); alert(`Your username is: \${username}`);</pre>	Your username: @mohsineli9

ARRAYS

Arrays

⇒ An **array** is a special variable that can hold **multiple values** in a single name.

- Array elements can be of **different data types**.
- Indexing starts at **0**.

Syntax:	Example:
<pre>let llistVariableName = ["item1", "item2"]</pre>	<pre>let fruits = ["Apple", "Banana", "Mango"];</pre>
Creating Arrays:	
<pre>let arr1 = ["Apple", "Banana", "Mango"]; // literal let arr2 = new Array("Apple", "Banana"); // constructor let arr3 = []; // empty array</pre>	
Accessing Array Elements:	
<pre>let fruits = ["Apple", "Banana", "Mango"]; console.log(fruits[0]); // Apple console.log(fruits[fruits.length - 1]); // Mango</pre>	
Changing Array Elements:	
<pre>fruits[1] = "Orange"; // changes Banana to Orange</pre>	

Common Array Properties:

Property	Description	Example
<code>.length</code>	Number of elements	<code>fruits.length</code> → 3

Example code:	Output:
<pre>let fruits = ["Apple", "Banana", "Mango"]; let length = fruits.length; console.log("Number of fruits:", length);</pre>	Number of fruits: 3

Common Array Methods:

Method	Description	Example
<code>push(item)</code>	Adds to end	<code>arr.push("Grapes")</code>
<code>pop()</code>	Removes last item	<code>arr.pop()</code>
<code>unshift(item)</code>	Adds to start	<code>arr.unshift("Lemon")</code>
<code>shift()</code>	Removes first item	<code>arr.shift()</code>
<code>concat(arr2)</code>	Joins arrays	<code>arr.concat(arr2)</code>
<code>join(separator)</code>	Array → string	<code>arr.join(", ")</code>
<code>slice(start, end)</code>	Copies part of array	<code>arr.slice(1, 3)</code>
<code>splice(start, deleteCount, ...items)</code>	Adds/removes items	<code>arr.splice(1, 1, "Kiwi")</code>
<code>indexOf(value)</code>	First index of value	<code>arr.indexOf("Apple")</code>
<code>lastIndexOf(value)</code>	Last index of value	<code>arr.lastIndexOf("Apple")</code>
<code>includes(value)</code>	Checks if exists	<code>arr.includes("Banana")</code>
<code>reverse()</code>	Reverses array	<code>arr.reverse()</code>
<code>sort()</code>	Sorts alphabetically	<code>arr.sort()</code>
<code>toString()</code>	Array → string	<code>arr.toString()</code>
<code>flat(depth)</code>	Flattens nested arrays	<code>[1, [2, 3]].flat()</code>

Multidimensional Arrays

Example code:	Output:
<pre>let matrix = [[1, 2, 3], [4, 5, 6]]; console.log(matrix[1][2]);</pre>	6

Looping Through Arrays:

Example code:	Output:
<pre>let fruits = ["Apple", "Banana", "Mango"]; // for loop for (let i = 0; i < fruits.length; i++) { console.log(fruits[i]); }</pre>	Apple Banana Mango

Practice Problem

🔗 **Problem1:** For a given array with marks of students → [85, 97, 44, 37, 76, 60] Find the average marks of the entire class.

Code:	Output:
<pre>let marks = [85, 97, 44, 37, 76, 60]; let sum = 0; for (let i = 0; i < marks.length; i++) { sum += marks[i]; } let average = sum / marks.length; console.log("Average Marks:", average);</pre>	Average Marks: 66.5

🔗 **Problem2:** For a given array with prices of 5 items → [250, 645, 300, 900, 50] All items have an offer of 10% OFF on them. Change the array to store final price after applying offer.

Code:	Output:
<pre>let prices = [250, 645, 300, 900, 50]; for (let i = 0; i < prices.length; i++) { let discount = prices[i] * 0.10; prices[i] = prices[i] - discount; } console.log("Final Prices after 10% OFF:", prices);</pre>	Final Prices after 10% OFF: (5) [225, 580.5, 270, 810, 45]

- ⌘ **Problem3:** Create an array to store companies → “Bloomberg”, “Microsoft”, “Uber”, “Google”, “IBM”, “Netflix”
- Remove the first company from the array
 - Remove Uber & Add Ola in its place
 - Add Amazon at the end

Code:	Output:
<pre>let companies = ["Bloomberg", "Microsoft", "Uber", "Google", "IBM", "Netflix"]; // a. Remove the first company companies.shift(); // b. Remove Uber & Add Ola in its place let uberIndex = companies.indexOf("Uber"); if (uberIndex !== -1) { companies.splice(uberIndex, 1, "Ola"); } // c. Add Amazon at the end companies.push("Amazon"); console.log(companies);</pre>	<pre>(6) ['Microsoft', 'Ola', 'Google', 'IBM', 'Netflix', 'Amazon']</pre>

FUNCTIONS

Functions

➔ A **function** is a block of reusable code that performs a specific task.

Define a Function:	Call a Function:
<pre>function function_name(){ console.log("Hello World!"); }</pre>	<pre>function_name();</pre>
Example code:	Output:
<pre>function greet(name) { console.log("Hello " + name); } greet("Mohsin");</pre>	Hello, Mohsin
Notes: Use function to define a function.	

Function Expression

⇒ A function can be stored in a variable.

Example code:	Output:
<pre>const greet = function(name) { console.log("Hello " + name); }; greet("Alif");</pre>	Hello Alif
Notes: Functions help make code modular and reusable .	

Arrow Functions

Example code:	Output:
<pre>const greet = (name) => { console.log("Hello " + name); }; greet("Jam");</pre>	Hello Jam

Function with Return Value

Example code:	Output:
<pre>function add(a, b) { return a + b; } let sum = add(5, 3); console.log(sum); // 8</pre>	8

Default Parameters

Example code:	Output:
<pre>function greet(name = "Guest") { console.log("Hello " + name); } greet(); // Hello Guest greet("Nihan"); // Hello Nihan</pre>	Hello Guest Hello Nihan

Rest Parameters

⇒ Allows a function to accept **any number of arguments**.

Example code:	Output:
<pre>function sum(...numbers) { let total = 0; for (let num of numbers) { total += num; } return total; } console.log(sum(1, 2, 3, 4)); // 10</pre>	10

Anonymous Functions

⇒ Functions without a name, usually used as arguments.

Example code:	Output:
<pre>setTimeout(function() { console.log("Hello after 2 seconds"); }, 2000);</pre>	Hello after 2 seconds

IIFE (Immediately Invoked Function Expression)

⇒ A function that runs **immediately** after it's defined.

Example code:	Output:
<pre>(function() { console.log("IIFE runs immediately"); })();</pre>	IIFE runs immediately

Scope in Functions

⇒ Variables inside a function are **local**.

⇒ Variables outside are **global**.

Example code:	Output:
<pre>let globalVar = 10; function test() { let localVar = 5; console.log(globalVar); // 10 console.log(localVar); // 5 } test(); console.log(localVar); // ❌ Error</pre>	10 5

Practice Problem

🔗 **Problem1:** Create a function using the “function” keyword that takes a String as an argument & returns the number of vowels in the string.

Code:	Output:
<pre>function countVowels(str) { let count = 0; let vowels = "aeiouAEIOU"; for (let i = 0; i < str.length; i++) { if (vowels.includes(str[i])) { count++; } } return count; } console.log(countVowels("Hello World"));</pre>	3

🔗 **Problem2:** Create an arrow function to perform the same task

Code:	Output:
<pre>const countVowels = (str) => { let count = 0; let vowels = "aeiouAEIOU"; for (let i = 0; i < str.length; i++) { if (vowels.includes(str[i])) { count++; } } return count; }; console.log(countVowels("Hello World"));</pre>	3

✂ **Problem3:** Create an arrow function to perform the same task

Code:	Output:
<pre>let numbers = [2, 4, 6, 8, 10]; numbers.forEach(function(num) { console.log(num * num); });</pre>	<pre>4 16 36 64 100</pre>

Problem4: We are given array of marks of students. Filter out of the marks of students that scored 90+.

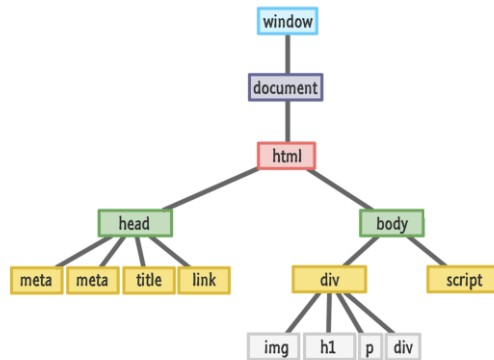
- Take a number n as input from user. Create an array of numbers from 1 to n.
- Use the reduce method to calculate sum of all numbers in the array.
- Use the reduce method to calculate product of all numbers in the array.

Code: Filter students with marks 90+	Output:
<pre>var marks = [85, 97, 44, 93, 76, 99, 91]; var highScorers = marks.filter(function(mark) { return mark > 90; }); console.log("Marks 90+ :", highScorers);</pre>	<pre>Marks 90+ : (4) [97, 93, 99, 91]</pre>
Code: Create array from 1 to n	Output:
<pre>var n = parseInt(prompt("Enter a number:")); var arr = []; for (var i = 1; i <= n; i++) { arr.push(i); } console.log("Array from 1 to n:", arr);</pre>	<pre>Array from 1 to n: (5) [1, 2, 3, 4, 5]</pre>
Code: Calculate product using reduce	Output:
<pre>var arr = [1,2,3,4,5]; var product = arr.reduce(function(accumulator, currentValue) { return accumulator * currentValue; }, 1); console.log("Product of numbers:", product);</pre>	<pre>Product of numbers: 120</pre>

DOM MANIPULATION

DOM Manipulation

- **DOM** (Document Object Model) represents the **HTML structure** of a webpage.
- JavaScript can **access, change, add, or remove HTML elements** using the DOM.



Accessing Elements

Method	Description	Example
<code>getElementById()</code>	Select element by id	<code>document.getElementById("myDiv")</code>
<code>getElementsByClassName()</code>	Select elements by class	<code>document.getElementsByClassName("myClass")</code>
<code>getElementsByTagName()</code>	Select elements by tag	<code>document.getElementsByTagName("p")</code>
<code>querySelector()</code>	Select first element matching CSS selector	<code>document.querySelector(".myClass")</code>
<code>querySelectorAll()</code>	Select all elements matching CSS selector	<code>document.querySelectorAll("p")</code>

Changing Content

Property / Method	Description	Example
<code>innerHTML</code>	Change HTML content	<code>element.innerHTML = "<div>Hello</div>"</code>
<code>textContent</code>	Change text content only	<code>element.textContent = "Hello"</code>
<code>innerText</code>	Change visible text	<code>element.innerText = "Hello"</code>

Changing Styles using DOM

Example code:	Output:	
	Before JS	After JS
<pre>let box = document.getElementById("Box"); box.style.backgroundColor = "red"; box.style.width = "200px"; box.style.border = "2px solid black";</pre>		

Changing Attributes

Method	Description	Example
<code>getAttribute()</code>	Get attribute value	<code>element.getAttribute("src")</code>
<code>setAttribute()</code>	Set attribute value	<code>element.setAttribute("src", "image.jpg")</code>
<code>removeAttribute()</code>	Remove an attribute	<code>element.removeAttribute("disabled")</code>

Example code:	Output:
<pre><div id="Box" title="Original Title">Box</div> <script> // Select the element let box = document.getElementById("Box"); // getAttribute() console.log("Title before:", box.getAttribute("title")); // setAttribute() box.setAttribute("title", "Updated Title"); console.log("Title After:", box.getAttribute("title")); // removeAttribute() box.removeAttribute("title"); console.log("Title After remove:", box.getAttribute("title")); </script></pre>	<pre>Title before: Original Title Title After: Updated Title Title After remove: null</pre>

Adding / Removing Elements

Example code:	Output:
<pre><div id="Box" title="Original Title">Box</div> <div id="Box2"></div> <script> // Create a new element let p = document.createElement("p"); p.textContent = "Hello World"; let div2 = document.getElementById("Box2"); // Append to body or another element div2.appendChild(p); // Remove an element let div = document.getElementById("Box"); div.remove(); </script></pre>	<pre><body> <div id="Box2"> <p>Hello World</p> </div> <script>...</script> </body></pre>

Event Handling

⇒ Respond to **user actions** like **clicks**, **mouse movements**, **keyboard input**.

Example code:	Output:
<pre><button id="myBtn">Click me</button> <script> let button = document.getElementById("myBtn"); button.addEventListener("click", function () { alert("Button clicked!"); }); </script></pre>	

➤ **Common Events:**

Event Name	Trigger Description
<code>click</code>	Mouse click
<code>mouseover</code>	Mouse moves over the element
<code>mouseout</code>	Mouse leaves the element
<code>keydown</code>	A key is pressed on the keyboard
<code>keyup</code>	A key is released on the keyboard
<code>submit</code>	A form is submitted

Traversing the DOM

Property	Description	Example
<code>parentNode</code>	Access parent element	<code>element.parentNode</code>
<code>children</code>	Access child elements	<code>element.children</code>
<code>firstChild</code>	First child node	<code>element.firstChild</code>
<code>lastChild</code>	Last child node	<code>element.lastChild</code>
<code>nextSibling</code>	Next sibling node	<code>element.nextSibling</code>
<code>previousSibling</code>	Previous sibling node	<code>element.previousSibling</code>

Example code: Traversing the DOM

```
<div id="parent">
  <p id="first">First paragraph</p>
  <p id="second">Second paragraph</p>
  <p id="third">Third paragraph</p>
</div>

<script>
  let secondPara = document.getElementById("second");

  // 1. parentNode
  console.log("Parent Node of second:", secondPara.parentNode);

  // 2. children
  console.log("Children of parent div:", secondPara.parentNode.children);

  // 3. firstChild
  console.log("First Child of parent div:", secondPara.parentNode.firstChild);

  // 4. lastChild
  console.log("Last Child of parent div:", secondPara.parentNode.lastChild);

  // 5. nextSibling
  console.log("Next Sibling of second paragraph:", secondPara.nextSibling);

  // 6. previousSibling
  console.log("Previous Sibling of second paragraph:", secondPara.previousSibling);
</script>
```

Output:

```
parentNode: <div id="parent">...</div>
children: HTMLCollection(3) [<p id="first">...</p>, <p id="second">...</p>, <p id="third">...</p>]
firstChild: <p id="first">First paragraph</p>
lastChild: <p id="third">Third paragraph</p>
nextSibling: <p id="third">Third paragraph</p>
previousSibling: <p id="first">First paragraph</p>
```

Example code: Event Handling

```
<button id="btnClick">Click Me</button><br><br>

<div id="hoverBox">Hover Here</div><br>

<input id="keyInput" type="text" placeholder="Type something..."><br><br>

<form id="myForm">
  <input type="text" placeholder="Your name">
  <button type="submit">Submit</button>
</form>

<script>
  // Elements
  const btnClick = document.getElementById("btnClick");
  const hoverBox = document.getElementById("hoverBox");
  const keyInput = document.getElementById("keyInput");
  const myForm = document.getElementById("myForm");

  // Actions
  const handleClick = () => {
    console.log("Button clicked!");
  };

  const handleMouseOver = () => {
    console.log("Mouse over the box!");
    hoverBox.style.backgroundColor = "lightgreen";
  };

  const handleMouseOut = () => {
    console.log("Mouse out of the box!");
    hoverBox.style.backgroundColor = "lightblue";
  };

  const handleKeyDown = (event) => {
    console.log("Key down:", event.key);
  };

  const handleKeyUp = (event) => {
    console.log("Key up:", event.key);
  };
</script>
```

```
const handleSubmit = (event) => {  
  event.preventDefault();  
  console.log("Form submitted!");  
};  
  
// Event Listeners  
btnClick.addEventListener("click", handleClick);  
hoverBox.addEventListener("mouseover", handleMouseOver);  
hoverBox.addEventListener("mouseout", handleMouseOut);  
keyInput.addEventListener("keydown", handleKeyDown);  
keyInput.addEventListener("keyup", handleKeyUp);  
myForm.addEventListener("submit", handleSubmit);  
</script>
```

Output:

Button clicked!

Mouse over the box!

Mouse out of the box!

Key down: b

Key up: b

Form submitted!

Example code: Changing Content

```
<div id="demo1">Original <b>HTML</b> content</div>
<div id="demo2">Original <b>HTML</b> content</div>
<div id="demo3">Original <b>HTML</b> content</div>

<button id="btnHTML">Change innerHTML</button>
<button id="btnTextContent">Change textContent</button>
<button id="btnInnerText">Change innerText</button>

<script>
  const demo1 = document.getElementById("demo1");
  const demo2 = document.getElementById("demo2");
  const demo3 = document.getElementById("demo3");

  const btnHTML = document.getElementById("btnHTML");
  const btnTextContent = document.getElementById("btnTextContent");
  const btnInnerText = document.getElementById("btnInnerText");

  // innerHTML changes HTML structure
  btnHTML.onclick = () => {
    demo1.innerHTML = "<i>Hello</i> <b>World</b>";
  };

  // textContent changes only text (HTML tags shown as plain text)
  btnTextContent.onclick = () => {
    demo2.textContent = "<i>Hello</i> <b>World</b>";
  };

  // innerText changes visible text (similar to textContent but respects CSS styles)
  btnInnerText.onclick = () => {
    demo3.innerText = "<i>Hello</i> <b>World</b>";
  };
</script>
```

Output: Before

Original **HTML** content
Original **HTML** content
Original **HTML** content

Change innerHTML

Change textContent

Change innerText

Output: After

Hello **World**

<i>Hello</i> World

<i>Hello</i> World

Change innerHTML

Change textContent

Change innerText

Example code: Accessing Elements

```
<div id="myDiv">This is a div with id "myDiv"</div>

<p class="myClass">Paragraph 1 with class "myClass"</p>
<p class="myClass">Paragraph 2 with class "myClass"</p>
<p>Paragraph 3 with no class</p>

<button id="btn">Select Elements</button>

<script>
  const btn = document.getElementById("btn");

  btn.onclick = () => {
    // 1. getElementById
    const divById = document.getElementById("myDiv");
    console.log("getElementById:", divById);

    // 2. getElementsByName
    const elemsByClass = document.getElementsByClassName("myClass");
    console.log("getElementsByName:", elemsByClass);

    // 3. getElementsByTagName
    const elemsByTag = document.getElementsByTagName("p");
    console.log("getElementsByTagName:", elemsByTag);

    // 4. querySelector (first match)
    const firstByQuery = document.querySelector(".myClass");
    console.log("querySelectorClass:", firstByQuery);

    // 5. querySelector by id
    const idByQuery = document.querySelector("#myDiv");
    console.log("querySelectorId:", idByQuery);

    // 6. querySelectorAll (all matches)
    const allByQuery = document.querySelectorAll("p");
    console.log("querySelectorAll:", allByQuery);
  };
</script>
```

Output Tracing Practice

Problem1.

Write the output of the following code	Answer
<pre><!DOCTYPE html> <html lang="en"> <body> <script> let x = "1"; const y = 2; let z = [1, 2, 3]; console.log("1" + x - y); console.log(parseInt(x)); let price = 9.99; console.log(price.toString() + x); z[z.length - 3] = Number(x) ** y; console.log(z[1]); console.log(typeof z); document.write(`\${typeof x}`); </script> </body> </html></pre>	Write the console output
	9 1 9.991 2 object
	Write the browser (document) output
	string

Problem2.

Write the output of the following code	Answer
<pre><!DOCTYPE html> <html lang="en"> <body> <script> let a = 5; let b = "10"; console.log(a + b); console.log(Number(b) - a); let arr = [a, b]; console.log(arr.length); arr[2] = a * 2; console.log(arr[2]); document.write(typeof b); </script> </body> </html></pre>	Write the console output
	510 5 2 10
	Write the browser (document) output
	string

Problem3.

Write the output of the following code	Answer
<pre> <!DOCTYPE html> <html lang="en"> <body> <script> const str = "20"; let num = 4; console.log(str / num); console.log(str + num); console.log(parseInt(str) + num); let data = ["x", "y", "z"]; data[data.length - 1] = "last"; console.log(data); document.write(typeof data); </script> </body> </html> </pre>	Write the console output
	<pre> 5 204 24 ["x", "y", "last"] </pre>
	Write the browser (document) output
	object

Problem4.

Write the output of the following code	Answer
<pre> <!DOCTYPE html> <html lang="en"> <body> <script> let price = 15.5; let quantity = "2"; console.log(price * quantity); console.log(price + quantity); console.log(Number(quantity) ** 2); let items = [1, 2, 3]; items[1] = price; console.log(items[1]); document.write(typeof price); </script> </body> </html> </pre>	Write the console output
	<pre> 31 15.52 4 15.5 </pre>
	Write the browser (document) output
	number

Problem5.

Write the output of the following code	Answer
<pre> <!DOCTYPE html> <html lang="en"> <body> <script> let x = "5"; let y = 3; console.log(x - y); console.log(x + y); console.log(parseFloat(x) + y); let nums = [10, 20]; nums[nums.length] = 30; console.log(nums.length); document.write(typeof y); </script> </body> </html> </pre>	Write the console output
	2 53 8 3
	Write the browser (document) output
	number

Problem6.

Write the output of the following code	Answer
<pre> <!DOCTYPE html> <html lang="en"> <body> <script> let a = "10"; let b = 5; console.log(a - b); console.log(a + b); console.log(a * "2"); console.log("5" + 3 - 2); let arr = [1, 2, 3]; arr[arr.length] = arr.length - 1; console.log(arr[arr[3]]); document.write(typeof (a + b - "2")); </script> </body> </html> </pre>	Write the console output
	5 105 20 51 3
	Write the browser (document) output
	number

🔗 **Problem7.**

Write the output of the following code	Answer
<pre> <!DOCTYPE html> <html lang="en"> <body> <script> let x; let y = null; let z = "5"; console.log(x + 1); console.log(y + 1); console.log(z - 2); console.log(z + 2); let data = [, 10, 20]; console.log(data[0]); document.write(x + "test"); data[0] = data[2] / data[1]; console.log(data[0]); document.write(typeof (x + "test")); </script> </body> </html> </pre>	Write the console output
	NaN 1 3 52 undefined 2
	Write the browser (document) output
	undefinedteststring

🔗 **Problem8.**

Write the output of the following code	Answer
<pre> <!DOCTYPE html> <html lang="en"> <body> <script> let a = "2"; let b = 2; console.log(a == b); console.log(a === b); console.log(a + b + "2"); console.log("2" + a * b); console.log((a + b) * "2"); let arr = [1, 2, 3]; arr.length = 5; arr[4] = arr.length; console.log(arr[arr[4] / 2]); console.log(null + "2" - 1); document.write(typeof (null + "2" - 1)); </script> </body> </html> </pre>	Write the console output
	true false 222 24 44 Undefined NaN
	Write the browser (document) output
	number

Problem9.

Consider the following HTML and JavaScript code:	Output of the browser is given:
<pre> <!DOCTYPE html> <html> <body> <div id="container"> <button id="btn1">btn 1</button> <button id="btn2">btn 2</button> </div> <script> const container = document.getElementById("container"); const btn1 = document.getElementById("btn1"); const btn2 = document.getElementById("btn2"); btn1.addEventListener("click", () => { const newPara = document.createElement("p"); newPara.textContent = "web"; container.appendChild(newPara); }); btn2.addEventListener("click", () => { const paragraphs = container.querySelectorAll("p"); if (paragraphs.length > 0) { const lastPara = paragraphs[paragraphs.length - 1]; container.removeChild(lastPara); } }); </script> </body> </html> </pre>	
What will be the output of empty spaces under the buttons after user perform following actions?	Write the output in the following sections
1) btn 1 is pressed 1 time	web
2) btn 2 is pressed 1 time and btn 1 is pressed 2 times	Web web
3) btn 2 is pressed 1 time	web
4) btn 1 is pressed 1 time and btn 2 is pressed 1 time	web
5) btn 1 is pressed 1 time	web web

Problem10.

Consider the following HTML and JavaScript code:	Output of the browser is given:
<pre> <!DOCTYPE html> <html> <body> <div id="container"> <button id="add">Add</button> <button id="remove">Remove</button> </div> <script> const container = document.getElementById("container"); const addBtn = document.getElementById("add"); const removeBtn = document.getElementById("remove"); addBtn.addEventListener("click", () => { const p = document.createElement("p"); p.textContent = "JS"; container.appendChild(p); }); removeBtn.addEventListener("click", () => { const paras = container.querySelectorAll("p"); if (paras.length > 0) { container.removeChild(paras[0]); } }); </script> </body> </html> </pre>	Add Remove
What will be the output of empty spaces under the buttons after user perform following actions?	Write the output in the following sections
1) Add pressed 2 times	JS JS
2) Add pressed 1 times, Remove pressed 1 time	JS JS
3) Add pressed 1 time, Remove pressed 2 times	JS
4) Remove pressed 1 time, Add pressed 2 times	JS JS
5) Remove pressed 1 times	JS

Problem11.

Consider the following HTML and JavaScript code:	Output of the browser is given:
<pre> <!DOCTYPE html> <html> <body> <div id="container"> <button id="btn1">Add HTML</button> <button id="btn2">Clear</button> </div> <script> const container = document.getElementById("container"); const btn1 = document.getElementById("btn1"); const btn2 = document.getElementById("btn2"); btn1.addEventListener("click", () => { const p = document.createElement("p"); p.textContent = "HTML"; container.appendChild(p); }); btn2.addEventListener("click", () => { const paras = container.querySelectorAll("p"); paras.forEach(p => container.removeChild(p)); }); </script> </body> </html> </pre>	Add HTML Clear
What will be the output of empty spaces under the buttons after user perform following actions?	Write the output in the following sections
1) Add HTML pressed 1 time	HTML
2) Add HTML pressed 3 times, Clear pressed 1 time	
3) Clear pressed 1 time, Add HTML pressed 2 time	HTML HTML
4) Add HTML pressed 2 times, Clear pressed 2 times	
5) Add HTML pressed 2 times, Clear pressed 1 times	

Problem12.

Consider the following HTML and JavaScript code:	Output of the browser is given:
<pre> <!DOCTYPE html> <html> <body> <div id="container"> <button id="btn1">Add</button> <button id="btn2">Remove Last</button> </div> <script> const container = document.getElementById("container"); const btn1 = document.getElementById("btn1"); const btn2 = document.getElementById("btn2"); btn1.addEventListener("click", () => { const p = document.createElement("p"); p.textContent = "Node"; container.appendChild(p); }); btn2.addEventListener("click", () => { const paras = container.querySelectorAll("p"); if (paras.length > 0) { const last = paras[paras.length - 1]; container.removeChild(last); } }); </script> </body> </html> </pre>	
What will be the output of empty spaces under the buttons after user perform following actions?	Write the output in the following sections
1) Add pressed 1 time	Node
2) Add pressed 2 times, Remove pressed 1 time	Node Node
3) Remove pressed 1 time, Add pressed 1 time	Node Node
4) Add pressed 3 times, Remove pressed 2 times	Node Node Node
5) Remove pressed 3 times	

Problem13.

Consider the following HTML and JavaScript code:

```
<!DOCTYPE html >
<html>
<body>

  <div id="container">
    <button id="PUSH">PUSH</button>
    <button id="POP">POP</button>
    <div id="status">Empty</div>
  </div>

  <script>
    const status = document.getElementById("status");
    const PUSH = document.getElementById("PUSH");
    const POP = document.getElementById("POP");

    let stack = [];
    let totalPushes = 0;

    function updateStatus() {
      const len = stack.length;
      if (len === 0) {
        status.innerHTML = "Empty";
      } else if (len === 1) {
        status.innerHTML = "First";
      } else if (len % 2 === 0) {
        status.innerHTML = "Even";
      } else {
        status.innerHTML = "Odd";
      }
    }

    function handlePush() {
      totalPushes++;
      stack.push(totalPushes);
      updateStatus();
    }

    function handlePop() {
      if (stack.length > 0) {
        stack.pop();
      }
      updateStatus();
    }

    PUSH.addEventListener("click", handlePush);
    POP.addEventListener("click", handlePop);

  </script>
</body>
</html>
```

Output of the browser is given:

PUSH POP

Empty

What will be the innerHTML content of the div under the buttons after a user performs the following actions? The actions are given sequentially. Write your answer under each action below (First one is done for you as an example).

1) Click PUSH 1 time:	First
2) Click PUSH 4 time:	Odd
3) Click POP 2 time:	Odd
4) Click PUSH 1 time:	Even
5) Click POP 3 time:	First
6) Click PUSH 1 time:	Even

Problem14.

Consider the following HTML code:	Output In Browser (Web Page)
<pre> <!doctype html> <html> <head> <style> #div1 { background-color: white; display: inline; position: absolute; z-index: 30; } #div2 { background-color: white; display: inline; position: absolute; z-index: 20; } #span1 { position: fixed; right: 0; } </style> </head> <body> <div id="div1">Box1</div> <div id="div2">Box2</div> Span1 <h1 id="h1" style="display: inline-block;">H1</h1> <div id="div3" style="display: inline;">Box3</div> </body> </html> </pre>	Box1Span1 H1Box3

Problem15.

Write the output of the following code	Answer
<pre> <!DOCTYPE html> <html> <body> <script> let userAge = "25"; let bonusPoints = "10"; let scores = [80, 85, 90, 95, 100]; console.log("Total:" + userAge + bonusPoints); document.write("Avg:" + userAge / 5 + "
"); let position = userAge - bonusPoints; console.log("Score" + position + " " + scores[position]); let isEligible = scores.length > 3; let bonus = isEligible ? bonusPoints : 0; document.write("Eligible: " + isEligible + "
"); let convertedAge = parseInt(userAge); console.log("Match: " + (userAge === convertedAge)); let removedScore = scores.pop(); document.write("Type:" + typeof removedScore + "
"); console.log("Count:" + scores.length); let scoreString = "98.7"; let decimal = parseFloat(scoreString); document.write("Sub:" + scoreString.substring(0, 2) + "
"); let undefinedVar; let nullVar = null; console.log("Undef: " + typeof undefinedVar); let outOfBounds = scores[10] "missing"; document.write("OOB: " + outOfBounds + "
"); </script> </body> </html> </pre>	Write the console output
	Total:2510 Score15 + undefined Match: false Count:4 Undef: undefined
	Write the browser (document) output
	Avg:5 Eligible: true Type:number Sub:98 OOB: missing

Code Writing Practice

✂ **Problem1:** You have given a task to create a product inventory management from for an e-commerce system that requires robust client-side validations. The form should allow staff to add new products. Write code for HTML form (**product.html**) with an external JavaScript file (**validations.js**) based on following guideline.

The form must collect and validate product details. 1) **Product Name** – Text Input, 2) **Price** – Number Input, 3) **Quantity in Stock** – Number Input, 4) **Category** – Dropdown Select, 5) **Expiry Date** – Date Input.

Your JavaScript validation should provide real-time feedback and display error messages using innerHTML based on following validation rule.

1. All fields are required
2. The price must be more then 0
3. Quantity in Stock must a number
4. Date must follow DD-MM-YYYY format
5. In Category, one options must be selected.

Make sure the form cannot be submitted without valid data

product.html

```
<!DOCTYPE html>
<body>
  <h2>Add New Product</h2>
  <form id="productForm">

    <label>Product Name:</label>
    <input type="text" id="productName">
    <span class="error"></span><br>

    <label>Price:</label>
    <input type="number" id="price">
    <span class="error"></span><br>

    <label>Quantity in Stock:</label>
    <input type="number" id="quantity">
    <span class="error"></span><br>

    <label>Category:</label>
    <select id="category">
      <option value="">--Select Category--</option>
      <option value="electronics">Electronics</option>
      <option value="clothing">Clothing</option>
    </select>
    <span class="error"></span><br>

    <label>Expiry Date:</label>
    <input type="date" id="expiryDate">
    <span class="error"></span><br>

    <br><br>
    <button type="submit">Add Product</button>
  </form>

  <script src="validations.js"></script>
</body>
</html>
```

validations.js

```
const form = document.getElementById("productForm");

const pName = document.getElementById("pName");
const price = document.getElementById("price");
const quantity = document.getElementById("quantity");
const category = document.getElementById("category");
const expiryDate = document.getElementById("expiryDate");

let error = document.querySelectorAll(".error");

function validateName() {
  if (pName.value.trim() === "") {
    error[0].innerHTML = "Product Name is required.";
    return false;
  }
  error[0].innerHTML = "";
  return true;
}

function validatePrice() {
  if (price.value === "") {
    error[1].innerHTML = "Product Price is required.";
    return false;
  } else if (Number(price.value) <= 0) {
    error[1].innerHTML = "Price must be more than 0.";
    return false;
  }
  error[1].innerHTML = "";
  return true;
}

function validateQuantity() {
  if (quantity.value === "") {
    error[2].innerHTML = "Quantity is required.";
    return false;
  } else if (isNaN(Number(quantity.value))) {
    error[2].innerHTML = "Quantity must be a number.";
    return false;
  }
  error[2].innerHTML = "";
  return true;
}

function validateCategory() {
  if (category.value === "") {
    error[3].innerHTML = "Please select a category.";
    return false;
  }
  error[3].innerHTML = "";
  return true;
}

function validateDate() {
  if (expiryDate.value === "") {
    error[4].innerHTML = "Expiry Date is required.";
    return false;
  }
  error[4].innerHTML = "";
  return true;
}

form.addEventListener("submit", function (event) {
  if (
    !validateName() ||
    !validatePrice() ||
    !validateQuantity() ||
    !validateCategory() ||
    !validateDate()
  ) {
    event.preventDefault();
  }
});
```

✂ **Problem2:** You are tasked with creating a **user registration form** for a website. Write code for HTML form (**registration.html**) with an external JavaScript file (**validations.js**) based on following guideline.

The form must collect and validate details. 1) **Full Name** – Text Input, 2) **Email Address** – Email Input, 3) **Password** – Password input, 4) **Confirm Password** – Password input, 5) **Gender** – Radio buttons, 6) **Date of Birth** – Date input

Your JavaScript validation should provide real-time feedback and display error messages using innerHTML based on following validation rule.

1. All fields are **required**
2. Email must be in valid email format
3. Password must be at least **8 characters**
4. Confirm Password must **match** the Password field
5. One **Gender** option must be selected
6. Date of Birth cannot be empty

Make sure the form cannot be submitted without valid data

registration.html

```
<!DOCTYPE html>
<body>
  <h2>User Registration</h2>
  <form id="regForm">

    <label>Full Name:</label>
    <input type="text" class="inputs">
    <span class="error"></span><br>

    <label>Email:</label>
    <input type="email" class="inputs">
    <span class="error"></span><br>

    <label>Password:</label>
    <input type="password" class="inputs">
    <span class="error"></span><br>

    <label>Confirm Password:</label>
    <input type="password" class="inputs">
    <span class="error"></span><br>

    <label>Gender:</label>
    <input type="radio" value="Male" class="inputs"> Male
    <input type="radio" value="Female" class="inputs"> Female
    <span class="error"></span><br>

    <label>DOB:</label>
    <input type="date" class="inputs">
    <span class="error"></span><br>

    <button type="submit">Register</button>
  </form>
  <script src="validations.js"></script>
</body>
</html>
```

validations.js

```
const form = document.getElementById("regForm");
const inputs = document.querySelectorAll(".inputs");
let error = document.querySelectorAll(".error");

form.addEventListener("submit", function (event) {
    let ok = true;

    if (inputs[0].value === "") {
        error[0].innerHTML = "Name is required.";
        ok = false;
    } else {
        error[0].innerHTML = "";
    }

    if (inputs[1].value === "") {
        error[1].innerHTML = "Email required.";
        ok = false;
    } else {
        error[1].innerHTML = "";
    }

    if (inputs[2].value === "") {
        error[2].innerHTML = "Password is required.";
        ok = false;
    } else if (inputs[2].value.length < 8) {
        error[2].innerHTML = "Min 8 characters.";
        ok = false;
    } else {
        error[2].innerHTML = "";
    }

    if (inputs[3].value === "") {
        error[3].innerHTML = "Confirm Password is required.";
        ok = false;
    } else if (inputs[2].value !== inputs[3].value) {
        error[3].innerHTML = "Passwords do not match";
        ok = false;
    } else {
        error[3].innerHTML = "";
    }

    if (!inputs[4].checked || !inputs[5].checked) {
        error[4].innerHTML = "Please select a gender.";
        ok = false;
    } else {
        error[4].innerHTML = "";
    }

    if (inputs[6].value === "") {
        error[5].innerHTML = "DOB is required.";
        ok = false;
    } else {
        error[5].innerHTML = "";
    }

    if (!ok) {
        event.preventDefault();
    } else {
        alert("Reg Success");
    }
});
```

Problem3: Write HTML and JavaScript code based on the scenario below:

You are a student in the Web Technologies course and have been assigned to create a simple web page. This web page should include two input fields: one for a teacher to enter a student's name, and another to enter the student's marks, along with a Submit button. Additionally, the page should have a table to display the students' information. The table must include a single header row with two column headings. The table must have two columns labeled "Name" and "Marks".

Before submitting the data, the web page must validate the inputs based on the following rules:

- The name field cannot be empty and must contain only letters (no numbers, spaces, or special characters).
- The marks field must be a number between 0 and 100 inclusive.

Clicking the Submit button should trigger the validation process.

If the inputs are valid, the student's name and marks should be added as a new row in the table below the input fields. This update should happen dynamically. Each valid submission adds a new row to the table.

Furthermore, each table row's background color should change based on the student's marks:

- If the marks are greater than 50, the row background should be green.
- If the marks are 50 or less, the row background should be red.

task.html

```
<!DOCTYPE html>
<html lang="en">
<head>
</head>
<body>
  <form id="form">
    <label>Student Name:</label>
    <input type="text" class="i">
    <span class="error"></span><br>

    <label>Student Number:</label>
    <input type="number" class="i">
    <span class="error"></span><br>

    <button type="submit">Submit</button><br><br><br>
  </form>

  <table id="table" border="1">
    <th colspan="2">Information</th>
    <tr>
      <th>Student Name</th>
      <th>Student Number</th>
    </tr>
  </table>
  <script src="script.js"></script>
</body>
</html>
```

validations.js

```
let form = document.getElementById("form");
let inputs = document.querySelectorAll(".i");
let e = document.querySelectorAll(".error");
let table = document.getElementById("table");

form.addEventListener("submit", function (event) {
  event.preventDefault();
  let ok = true;

  // validate name
  if (inputs[0].value === "") {
    e[0].innerHTML = "Please enter a name";
    ok = false;
  } else if (/\\d/.test(inputs[0].value) || inputs[0].value.includes("@") ||
    inputs[0].value.includes(" ")) {
    e[0].innerHTML = "Name cannot contain numbers, @, #, $, or spaces";
    ok = false;
  } else {
    e[0].innerHTML = "";
  }

  // validate number
  let num = Number(inputs[1].value);
  if (inputs[1].value === "") {
    e[1].innerHTML = "Number cannot be empty";
    ok = false;
  } else if (isNaN(num) || num > 100 || num < 0) {
    e[1].innerHTML = "Number must be a number between 0 and 100";
    ok = false;
  } else {
    e[1].innerHTML = "";
  }

  // add row if valid
  if (ok) {
    let tr = document.createElement("tr");
    tr.innerHTML = `<td>${inputs[0].value}</td><td>${inputs[1].value}</td>`;
    table.appendChild(tr);

    if (inputs[1].value < 50) {
      tr.style.backgroundColor = "red";
    } else {
      tr.style.backgroundColor = "green";
    }

    inputs[0].value = "";
    inputs[1].value = "";
  }
});
```

✂ **Problem4:** You are asked to create an **event booking form** for a conference website. Write code for HTML form (**booking.html**) with an external JavaScript file (**validations.js**) based on following guideline.

The form must collect and validate details. 1) **Full Name** – Text Input, 2) **Email Address** – Email Input, 3) **Number of Tickets** – Number input, 4) **Event Type** – Dropdown (Workshop, Seminar, Networking), 5) **Event Date** – Date input, 6) **Agree to Terms** – Checkbox.

Your JavaScript validation should provide real-time feedback and display error messages using innerHTML based on following validation rule.

1. All fields are **mandatory**
2. Email must be valid
3. Number of Tickets must be a **positive number**
4. Event Type must have **one option selected**
5. Event Date cannot be empty
6. Checkbox **must be checked** to allow submission

Make sure the form cannot be submitted without valid data

booking.html

```
<!DOCTYPE html>
<body>
  <h2>Event Booking Form</h2>
  <form id="regForm">

    <label>Full Name:</label>
    <input type="text" class="inputs">
    <span class="error"></span><br>

    <label>Email:</label>
    <input type="email" class="inputs">
    <span class="error"></span><br>

    <label>Number of Tickets:</label>
    <input type="number" class="inputs">
    <span class="error"></span><br>

    <label>Event Type:</label>
    <select class="inputs">
      <option value="Workshop">Workshop</option>
      <option value="Seminar">Seminar</option>
      <option value="Networking">Networking</option>
    </select>
    <span class="error"></span><br>

    <label>DOB:</label>
    <input type="date" class="inputs">
    <span class="error"></span><br>

    <label>Agree Terms & Condition:</label>
    <input type="checkbox" class="inputs">
    <span class="error"></span><br>

    <br><br>
    <button type="submit"> Book Now</button>
  </form>

  <script src="validations.js"></script>
</body>
</html>
```

validations.js

```
const form = document.getElementById("regForm");
const inputs = document.querySelectorAll(".inputs");
let error = document.querySelectorAll(".error");

form.addEventListener("submit", function (event) {
  let ok = true;

  if (inputs[0].value === "") {
    error[0].innerHTML = "Name is required.";
    ok = false;
  } else {
    error[0].innerHTML = "";
  }

  if (inputs[1].value === "") {
    error[1].innerHTML = "Email required.";
    ok = false;
  } else {
    error[1].innerHTML = "";
  }

  const ticketNum = Number(inputs[2].value);
  if (inputs[2].value === "") {
    error[2].innerHTML = "Tickets is required.";
    ok = false;
  } else if (isNaN(ticketNum) || ticketNum <= 0) {
    error[2].innerHTML = "Must be a positive number.";
    ok = false;
  } else {
    error[2].innerHTML = "";
  }

  if (inputs[3].value === "") {
    error[3].innerHTML = "Type required.";
    ok = false;
  } else {
    error[3].innerHTML = "";
  }

  if (inputs[4].value === "") {
    error[4].innerHTML = "DOB is required.";
    ok = false;
  } else {
    error[4].innerHTML = "";
  }

  if (!inputs[5].checked) {
    error[5].innerHTML = "Must agree to terms.";
    ok = false;
  } else {
    error[5].innerHTML = "";
  }

  if (!ok) {
    event.preventDefault();
  } else {
    alert("Booking successful");
  }
});
```


Fill in the Blanks Practice

✂ **Problem1:** Choose the correct word from the list below to complete each sentence.

static	protocol	client	dynamic	design	input
search	server	response	render	IP	request
URL	error	HTTP	HTML		

1. When a student enters a website URL into their browser, the browser translates the domain name into an IP address to find the correct server.
2. The browser acts as a client sending requests to the server and displaying the responses to the user.
3. The university's web server responds to requests by sending HTML documents, which are then rendered into visual pages by the browser.
4. Technologies like HTML, CSS, and JavaScript are used to design style, and add interactivity to web pages.
5. The communication between a browser and a server typically follows the HTTP protocol, which defines how messages are formatted and transmitted.
6. Every web interaction involves at least two main components: the client which initiates the request, and the server, which processes and responds.
7. The search bar in a browser is where users can enter a web address or URL.
8. A webpage that changes its content automatically based on user actions or data is known as a dynamic webpage.
9. If the server is unavailable or the page doesn't exist, the user might see a error code, indicating what kind of error occurred.
10. A HTTP request is sent by the browser to retrieve resources such as images, scripts, or data from the server.

✂ **Problem2:** Write yes or no in the blank, based on the question

Question	action	method	enctype	novalidate
Does this attribute define the URL to submit form data?	Yes	No	No	No
Does this attribute define the HTTP method (GET/POST)?	No	Yes	No	No
Does this attribute specify the encoding type for form data?	No	No	Yes	No
Does this attribute disable form validation?	No	No	No	Yes

✂ **Problem3:** Write **Yes** or **No** in the blank, based on each statement.

Statement	Answer
A browser acts as a client , sending requests to the server and displaying responses.	Yes
The communication between browser and server always uses the HTTP protocol .	No
A dynamic webpage updates its content automatically based on user actions or data.	Yes
DNS translates human-readable domain names into IP addresses.	Yes
NAT (Network Address Translation) allows multiple devices to share a single public IP.	Yes

✂ **Problem4:** Match the attribute with its behavior

Form Attribute	Behavior/Description	matching
action	A. Prevents browser validation of the form	B
method	B. Specifies the URL where form data is sent	D
enctype	C. Determines how form data is encoded for submission	C
novalidate	D. Chooses GET or POST for sending form data	A
multiple	E. Allows selecting more than one file at a time	E

MCQ Practice

1. **Which HTTP request method is used to submit data to be processed to a server?**
 - a) Get
 - b) **POST**
 - c) PUT
 - d) DELETE
2. **What is XHTML?**
 - a) **A Stricker version of HTML**
 - b) An extension of HTML
 - c) A document type delimitation
 - d) An image formats
3. **Which of the following is not a feature of HTTP?**
 - a) **Connection oriented**
 - b) Stateless
 - c) Request response protocol
 - d) Connectionless
4. **The DOM is used to _____**
 - a) Structure HTML documents only
 - b) **Dynamically access and update the content and style of a document**
 - c) Replace XML tags
 - d) Encode URLs
5. **What does DHTML stand for?**
 - a) **Dynamic Hyper Text Markup Language**
 - b) Dynamic High Text Markup Language
 - c) Direct Hyper Text Markup Language
 - d) Document Hyper Text Markup Language
6. **Which protocol is primarily used for secure communications over the internet?**
 - a) HTTP
 - b) FTP
 - c) **HTTPS**
 - d) SMTP
7. **Which is the following XML naming rules**
 - a) Names can start with a number
 - b) Names can start with a punctuation character
 - c) **Names cannot contain space**
 - d) Names cannot use any alphabetic character
8. **In HTML which attribute is used to specify an image source?**
 - a) link
 - b) **src**
 - c) href
 - d) ref

9. Which of the HTML elements is used to create a checkbox in forms?
- a) <button type = 'checkbox'>
 - b) <input type = 'button'>
 - c) <input type = 'checkbox'>
 - d) <button type = 'check'>
10. What does the get HTTP request method do?
- a) Delete data from the server
 - b) Retrieves data from the server
 - c) Sends data to be processed to a server
 - d) Updates existing server data
11. Which organization is responsible for stabilizing web standards?
- a) IETF
 - b) W3C
 - c) HTML5 Consortium
 - d) ISO
12. Network of networks that consists of millions of private, public, academic, business, and government networks is called web.
- a) True
 - b) false
13. Which tag is used to create a hyperlink in HTML?
- a) <a>
 - b) <link>
 - c) <url>
 - d) <href>
14. Which element in HTML is used to create an unordered list?
- a)
 - b)
 - c)
 - d) <list>
15. In XHTML, all elements must be _____
- a) uppercase
 - b) lowercase
 - c) bold
 - d) italic
16. The Document Object Model (DOM) represents HTML and XML documents as what type of structure?
- a) Table
 - b) Matrix
 - c) Tree
 - d) List
17. What does XML stand for?
- a) Extensive Markup Language
 - b) Extra Markup Language
 - c) Extensible Markup Language
 - d) Extended Markup Language
18. What is the main purpose of the HTTP protocol?
- a) To transfer files between servers
 - b) To authenticate servers
 - c) To exchange hypertext information between client and server
 - d) To format HTML documents
19. Which HTML input type is used for a password field?
- a) <input>
 - b) <input type="number">
 - c) <input type="password">
 - d) <input type="hidden">

20. In a URL, what does the 'path' specify?
- a) The domain name
 - b) The protocol
 - c) The file location on the server
 - d) The fragment identifier
21. What feature makes DHTML different from HTML?
- a) Static content
 - b) Dynamic interactivity
 - c) Hypertext links
 - d) Structured formatting
22. Which HTML element is used to represent a numbered list?
- a) <list>
 - b)
 - c)
 - d) <nulls>
23. Which tag is used for inserting a line break in HTML?
- a) <lb>
 - b) <line>
 - c)

 - d) <newline>
24. What is the purpose of the <form> element in HTML?
- a) Display images
 - b) Define a form to collect user input
 - c) Structure the layout
 - d) Insert links
25. In a client-server model, who initiates the request?
- a) Server
 - b) Client
 - c) Peer
 - d) Router
26. URI stands for:
- a) Uniform Resource Identifier
 - b) Universal Resource Identifier
 - c) Uniform Resource Interlink
 - d) Unified Resource Integration
27. The HTTP protocol is designed to permit intermediate network elements to improve or enable communications between clients and servers and it is a stateless protocol.
- a) True
 - b) False
28. What is the correct HTML for creating a hyperlink?
- a) aiub.edu
 - b) <a>http://www.aiub.edu
 - c) AIUB
 - d) AIUB
29. Which HTTP request method is used to retrieve data from a server?
- a) GET
 - b) POST
 - c) PUT
 - d) DELETE

30. The <head> sections of an HTML document is used to _____
- a) Display content
 - b) Define metadata
 - c) Structure page layout
 - d) Define DOM
31. In HTML5, the <canvas> element is used for _____
- a) Creating table
 - b) Drawing graphics
 - c) Embedding videos
 - d) Defining forms
32. Which attribute is necessary for file upload in HTML forms?
- a) method="post"
 - b) enctype="multipart/form-data"
 - c) action="file_upload"
 - d) name="file"
33. Which of these elements are all table elements?
- a) <table><head><tfoot>
 - b) <table><tr><td>
 - c) <table>
 - d) <table><head><body>
34. Which of the following is used to include the contents of one file into another?
- a) include()
 - b) header()
 - c) append()
 - d) load()
35. In the URL: "<https://www.example.com/page.html?id=123>", what does "id=123" represent?
- a) Anchor
 - b) File path
 - c) Query Parameter
 - d) Domain Name
36. Which one of the following statements is false about form handling?
- a) Any amount of data can be supplied from a form using the POST method
 - b) The HTTP GET method should never be used for sending passwords or other sensitive information
 - c) The HTTP POST requests are never cached, and we cannot bookmark them
 - d) The POST method is limited to 1024 bytes
37. Which of the following is used to return the length of an array in JS?
- a) .size
 - b) .count
 - c) .len
 - d) .length

38. Which of the following is true about the id attribute in HTML?
- a) An element can have multiple id values
 - b) The id attribute must be unique within a single HTML document**
 - c) The id attribute is only used for form elements
 - d) The id attribute is optional and cannot be used for styling
39. A router is configured to allow multiple devices in a home network to share a single public IP address. Which technology is being used here?
- a) HTTP
 - b) DNS (Domain Name System)
 - c) NAT (Network Address Translation)**
 - d) TCP/IP
40. What is the purpose of the <head> section in an HTML document?
- a) To define the visible content of the body
 - b) To store metadata and links to stylesheets or scripts**
 - c) To contain user input forms
 - d) To display the page title on the body
41. You are creating a webpage for a travel blog. On the blog, there are links to external websites like booking.com and tripadvisor.com. You want these external links to open in a new tab so that users don't lose their place on your blog. Which target attribute value should you use in the <a> tag to achieve this behavior?
- a) blank b) parent c) self d) top
42. What is an attribute in HTML?
- a) A function inside an HTML tag
 - b) A property that provides additional information about an element**
 - c) A type of CSS style
 - d) A separate HTML tag
43. JavaScript was invented by
- a) Brendon Eich, 1995**
 - b) Tim Berners-Lee, 1990
 - c) Yukihiro Matsumoto, 1990
 - d) Bjarne Stroustrup, 1979
44. What is the primary role of a web HTTP server?
- a) To act as a database server
 - b) To serve static and dynamic web pages**
 - c) To manage version control for the project
 - d) To provide a graphical user interface for the server
45. <https://example.com:7000/products?id=123>. What is the port used in this URI?
- a) https
 - b) 8080
 - c) 7000**
 - d) 80

46. Which attribute is used to merge two or more columns in an HTML table?
- a) rowspan
 - b) colspan
 - c) merge
 - d) width
47. You are styling a box, and you need the content to be spaced from the border by 15px on all sides. Which CSS property will you use?
- a) margin
 - b) padding
 - c) border
 - d) width
48. In a form, you want to target only the input fields that are of type "text". Which selector will be the best choice?
- a) `input[type="text"]{}`
 - b) `input.text {}`
 - c) `input[type=text]{}`
 - d) `input:checkbox{}`
49. You are designing a webpage and want to create space between two elements, so they don't appear too close to each other. However, you want to avoid changing the size or internal layout of either element. Which CSS property should you use to add space between the elements without affecting their internal layout or size?
- a) padding
 - b) margin
 - c) border
 - d) width
50. To parse a string and return the first integer, we use the `parseInt()` method in JavaScript. The output of `parseInt("20 min")` will be:
- a) 20
 - b) NaN
 - c) Undefined
 - d) Type error
51. Suppose you are required to create variables whose scope are defined within a block of code. JavaScript allows you to create block scopes using:
- a) `let`
 - b) `const`
 - c) `var`
 - d) Both `let` and `const`
52. A developer is building a social media application. Which HTTP method should be used to create a new post?
- a) GET
 - b) POST
 - c) PUT
 - d) DELETE

53. Which characteristic of HTTP makes possible to ensure that the server does not store any information about the client's previous requests. Which characteristic of HTTP makes this possible?
- a) Connectionless
 - b) Stateless
 - c) Secure
 - d) Secureless
54. What is the output of `const arr = [1,2]; arr.push(3)`
- a) [1,2]
 - b) [3]
 - c) [1,2,3]
 - d) [0,1,2,3]
55. To store and hold more than one item of data at the same time, an array could be a perfect arrangement. JavaScript requires the following syntax to declare an array:
- a) `var arrayname = new Array();`
 - b) `var arrayname = new array();`
 - c) `var myArray = [size];`
 - d) `var myArray = new array[size];`
56. What is the output of the snippet below
- ```
var fruits = ["Apple", "Orange", "Apple", "Mango"]; console.log(typeof(fruits));
```
- a) Array
  - b) String
  - c) Number
  - d) Object
57. Which one is correct to convert the date to string using this standard, `var d = new Date();`
- a) `Date.now();`
  - b) `d.toISOString();`
  - c) `d.getTime();`
  - d) `d.parseString();`
58. How do you select all elements with class "item" as a static NodeList?
- a) `document.querySelectorAll(".item")`
  - b) `document.getElementsByClassName("item")`
  - c) `document.getItemsByClass("item")`
  - d) `document.find(".item")`
59. You're tasked with styling a webpage that should have a consistent look across multiple pages. Which CSS method would you choose to ensure styles are applied universally across the site?
- a) Inline CSS
  - b) Internal CSS
  - c) External CSS
  - d) Embedded CSS

60. What is the correct syntax to add a click event to a button with ID "myBtn"?

- a) `myBtn.addEventListener("click", myFunction);`
- b) `document.getElementById("myBtn").onClick(myFunction);`
- c) `document.querySelector("myBtn").addEvent("click", myFunction);`
- d) `myBtn.click("myFunction");`

61. Which selector would you use to apply styles to an element with the id "main-header"?

- a) `#main-header {}`
- b) `.main-header {}`
- c) `main-header {}`
- d) `*main-header {}`

62. What is output of this code:

```
let x = 5;
console.log(x);
```

- a) 5
- b) undefined
- c) ReferenceError
- d) null

63. You want to ensure that an element takes up the entire width of its container and starts on a new line. What display property will you use?

- a) inline-block
- b) inline
- c) block
- d) flex

64. What is output of the following code:

```
const arr = [1, 2];
arr[100] = 3;
console.log(arr.length);
```

- a) 2
- b) 3
- c) 100
- d) 101

65. What is the correct way to display an image in HTML7?

- a) ``
- b) ``
- c) `<picture src="pic.jpg">`
- d) `<pic src="pic.jpg">`

66. What is the purpose of the id attribute in HTML?

- a) To define a heading
- b) To style an element using CSS
- c) To uniquely identify an element
- d) To create a hyperlink

67. Which of the following is true about XML?

- a) It is used only for web design
- b) It is used to style an element
- c) It is a programming language
- d) It is used to structure data

68. What does DHTML stand for?

- a) Dynamic Hyper Text Markup Language
- b) Direct Hyper Text Markup Language
- c) Dynamic High Text Markup Language
- d) Document Hyper Text Markup Language

69. Which type of HTML element is used to create a checkbox in forms?

- a) `<input type="checkbox">`
- b) `<button type="check">`
- c) `<input type="button">`
- d) `<check>`

70. URI stands for:

- a) Universal Resource Identify
- b) Uniform Resource Identifier
- c) Unified Resource Identify
- d) Universe Resource Identifier

71. The HTTP protocol is designed to permit intermediate network elements to improve or enable communications between clients and servers and it is a stateless protocol.

- a) True
- b) False

72. What does `Number("123abc")` return?

- a) 123
- b) 0
- c) NaN
- d) null

73. Which of these elements are all `<table>` elements?

- a) `<table><tr><tt>`
- b) `<table><tr><td>`
- c) `<table><head><tfoot>`
- d) `<thead><body><tr>`

74. How do you access an element by its ID in JavaScript?
- a) document.getElementById()
  - b) document.getElementById()
  - c) document.getElementById()
  - d) document.ElementById()
75. Which function is used to validate numeric input values in JavaScript?
- a) isNaN()
  - b) isNumber()
  - c) validateNumber()
  - d) checkNum()
76. Which event is most likely to trigger form validation just before the form is submitted, ensuring all fields are correct in JavaScript?
- a) onkeypress
  - b) onaction
  - c) onclick
  - d) onsubmit
77. What is the default value of an undeclared variable?
- a) null
  - b) undefined
  - c) 0
  - d) false
78. What is the result of  $(0.1 + 0.2 === 0.3)$ ?
- a) true
  - b) false
  - c) undefined
  - d) NaN

## Lecture Video Link

1. Web Tech (Midterm) — JavaScript: <https://youtu.be/Sv94XPtsN90?si=vvUZSf9GMRblbwvZ>